

CITE-seq Preprocessing Tutorial (NIPS / `cite_nips`)

A standalone walkthrough of the minimal preprocessing pipeline in `nips_preprocess_clean.ipynb`. It turns one paired CITE-seq object into two clean, analysis-ready matrices — **RNA (GEX)** and **protein (ADT)** — with sensible defaults, while keeping the `global` / `by_batch` and HVG-mode knobs exposed as functions so the same code serves both batch-integration benchmarks and downstream modeling (e.g. RNA → protein prediction). Where the right default depends on the task, the tutorial says so.

0. What you start with and what you get

The input `fullbatch.h5ad` is a **paired** object: every cell carries both transcriptome and surface-protein measurements stored side by side in the same matrix. Two pieces of metadata make this work:

- `adata.var["feature_types"]` — labels each column (feature) as `"GEX"` or `"ADT"`.
- `adata.layers["counts"]` — raw integer counts for **all** features.

The whole pipeline reads from `layers["counts"]`, never from `adata.X`, so it doesn't matter what state `.X` happens to be in when you load the file.

Output	Features	Final <code>.X</code>	Extra layers
<code>adata_rna_proc</code>	selected GEX HVGs	scaled log-norm (no clipping by default)	<code>counts</code> , <code>lognorm</code>
<code>adata_adt_proc</code>	all ADT proteins	CLR-normalized	<code>counts</code> , <code>clr</code>

Both objects contain the **same cells in the same order**, so they stay paired and can be concatenated feature-wise or fed jointly to a multimodal method.

1. Load and clean

```
python
adata = sc.read_h5ad(path)
adata.var_names_make_unique()
```

Two checks are worth keeping as guard rails:

1. **A real counts layer exists.** Everything downstream (HVG selection, CLR) assumes raw counts. If `layers["counts"]` is missing or has already been log-transformed, the

results are silently wrong.

2. **Counts are non-negative integers.** A quick min/max/integer check on a sub-block catches the common failure where someone saved normalized values into the counts layer.

The optional **major cell-type grouping** collapses the ~45 fine labels into coarser lineages. Two granularities are provided, and you pick based on how fine a stratification you need:

- **7-class** (`cell_library`): `T cell`, `monocyte`, `B cell`, `NK/ILC`, `DC`, `Progenitor`, `Erythroid`. The fuller picture — keeps rare compartments (dendritic, progenitor, erythroid) as their own groups.
- **4-class** (`cell_library2`): `T cell`, `B cell`, `NK`, `monocyte`. A coarser immune-lineage view that folds away the rare populations; useful when some groups are too small to give stable per-group statistics, or when you only need a broad-strokes split.

Neither is needed for normalization — they're for downstream analysis that stratifies by cell type. Anything not in the chosen dictionary maps to `Other`, so check that printout to confirm the map didn't silently drop labels.

2. The two modalities need different normalizations

This is the central idea. RNA and protein counts have different statistics, so they get different treatments.

RNA (GEX): library-size normalization

Transcript counts scale with sequencing depth per cell. The standard fix is:

```
counts → normalize_total(target_sum=1e4) → log1p → scale (z-score)
```

- `normalize_total` puts every cell on the same total count (CP10K) to remove depth differences.
- `log1p` stabilizes variance and tames the long right tail of highly expressed genes.
- `scale` z-scores each gene so no single high-variance gene dominates PCA.
- **Clipping is off by default** (`max_value=None`). Hard-clipping z-scores at ± 10 is the scanpy/Seurat/scIB convention for *unsupervised embedding* benchmarks, where a few outlier cells can rotate the PCA axes. It is not the default here — see the note below on when to turn it on.

Protein (ADT): centered log-ratio (CLR)

ADT panels are small (tens of proteins) and **compositional** — antibody capture is a shared, limited resource, so a protein's count is only meaningful *relative to the other proteins in that*

cell. Library-size log-norm is the wrong tool here, because total ADT count is not a clean nuisance variable. The community standard (Seurat WNN, muon) is **CLR**:

```
counts → CLR (no scaling, no clipping)
```

So ADT uses CLR, not the RNA-style `normalize_total/log1p` path — and the reason is statistical, not just convention. Applying library-size normalization to a panel of a few dozen proteins would divide out part of the very signal you care about, because total ADT count is itself partly biological (some cell types simply express more surface protein), not a clean technical nuisance like RNA depth. CLR instead expresses each protein *relative to a within-group reference*, which is the correct model for compositional measurements. Section 5 covers the mechanics and the choices that matter when ADT is a prediction target.

3. HVG selection (function: `select_hvg`)

```
python  
  
hvg = select_hvg(adata_gex, n_hvg=2000, mode="global", batch_key="batch")
```

Key points:

- **Run on raw counts with `flavor="seurat_v3"`.** Seurat v3 expects counts (it models the mean-variance relationship internally), which is why HVGs are selected *before* normalization. This needs the `scikit-misc` package installed.
- **The default here is `mode="global"` — and the right choice depends on the task.** `batch_aware` ranks variability within each batch then merges; its whole purpose is to *avoid* genes that look variable only because of batch effects, which is exactly what you want when batch integration is the objective (the scIB benchmark default). But when batch is **not** the target — e.g. cross-modal prediction (RNA → protein), where you simply want the most informative genes — that purpose disappears, and ranking by raw variability (`global`) is the more sensible default: it keeps the most predictive genes without second-guessing batch. Use `batch_aware` when batch correction is the actual goal; use `global` otherwise.
- **`global` also sidesteps a `seurat_v3` edge case.** Because it doesn't pass `batch_key`, it can't trip the "gene is all-zero within one batch" error that batch-aware HVG occasionally hits on small batches.
- **HVGs are GEX-only.** ADT proteins are never subsetted — the panel is small and every protein is informative, so all of them are kept.

For prediction tasks specifically: consider a larger `n_hvg` (3000–5000) or even no HVG filter at all, letting a regularized model do feature selection. A gene with modest

overall variance can still be a strong predictor of a particular protein, and HVG filtering would drop it.

Split-aware caveat. If you create train/test splits, select HVGs on the **training cells only** and apply that gene list to the test set. Selecting HVGs on the full data before splitting leaks information across the split.

4. Scaling (function: `scale_expression`)

```
python
```

```
X = scale_expression(adata, mode="global", batch_key="batch", max_value=None)
```

`global` z-scores each gene across **all** cells. `by_batch` z-scores each gene **independently within each batch**, so every batch is centered to mean 0 and unit variance on its own.

Mode	Effect	When to use
<code>global</code>	one mean/variance per gene	default; a single, consistent feature definition across all cells (and the basis for fit-on-train / transform-on-test in prediction)
<code>by_batch</code>	per-batch mean/variance	a mild, geometry-level batch correction baked into preprocessing

The choice is not cosmetic: `by_batch` removes first-moment (mean) batch differences before any correction method runs, which changes what downstream metrics measure. Treat the scaling mode as a deliberate experimental axis, not an afterthought — and report which one you used.

Clipping is off by default (`max_value=None`), for both RNA and ADT. Pass a finite value (e.g. `max_value=10`) only if you specifically want the unsupervised-embedding behavior where extreme z-scores are capped. For most supervised / prediction use a regularized model handles outliers better than a hard cap; if extreme values are a real concern, a data-driven winsorize (e.g. 99.5th percentile, fit on the training cells only) is preferable to a fixed ± 10 .

5. CLR in detail (function: `clr_transform`)

Why CLR at all

Surface-protein counts from an antibody panel are **compositional**: the antibodies compete for a shared, finite amount of capture per cell, so a protein's raw count is only interpretable

relative to the other proteins measured in that same cell. Two cells can have very different total ADT counts purely from technical capture efficiency, yet identical *relative* protein profiles. CLR is the standard fix for exactly this situation — it re-expresses each value as a log-ratio against a per-group geometric-mean reference, which cancels the shared multiplicative factor and leaves the relative composition. That's why plain library-size log-norm (the RNA recipe) is the wrong tool: it would treat total ADT as pure nuisance and divide out signal.

The implementation matches `muon.prot.pp.clr` exactly:

```
clr(x) = log1p( x / exp( mean_axis( log1p(x) ) ) )
```

`exp(mean(log1p(x)))` is the geometric-mean-style reference; dividing by it puts each value on a ratio scale, and the outer `log1p` makes those ratios additive and roughly symmetric. Because it's a ratio to a reference, CLR naturally produces a compact, centered distribution — which is why **no clipping is needed** for ADT.

The `axis` choice

<code>axis</code>	Normalizes...	Equivalent to	Self-contained per cell?
<code>0</code> (default)	each protein across all cells	muon default, Seurat <code>margin=2</code>	No — reference uses all cells
<code>1</code>	each cell across its proteins	canonical compositional CLR	Yes — reference is the cell itself

`axis=0` is the prevailing single-cell convention. `axis=1` is the textbook compositional definition (geometric mean over the components of one cell's profile). The "self-contained" column matters the moment you have a train/test split — see the prediction note below. Pick one and keep it fixed across datasets and splits so results stay comparable.

Post-CLR z-scoring (`scale_adt=True`) is available but **off by default**, since CLR already yields a well-behaved range.

If ADT is a prediction target: predict CLR, or predict raw counts?

When the ADT side is the *output* of a model (RNA → protein), the normalization choice becomes a modeling decision, and the two reasonable options trade off cleanly:

Predict CLR (or log-normalized) values — good for generic regressors. A continuous, symmetric, roughly homoscedastic target is what an MSE-trained model (linear, ridge, MLP, gradient boosting) wants. Raw counts are skewed, overdispersed, and depth-dependent, so MSE on raw counts is poorly behaved. CLR additionally removes the per-cell capture confound, so the model learns *relative* protein levels rather than total-capture artifacts. The catch is the `axis` choice above: predicting **axis=1** CLR is clean because each

cell's target depends only on that cell; predicting **axis=0** CLR makes the target depend on the whole cell set, so to avoid leakage you must compute the per-protein reference on the **training cells only** and apply it to test (treat it like a fitted scaler).

Predict raw counts — good for count-aware probabilistic models. If the model has a count likelihood (negative-binomial / Poisson — e.g. totalVI-style decoders), predict raw counts directly and pass library size as an offset/covariate. This respects the actual data-generating process and the mean-variance relationship, keeps proteins independent (no compositional coupling between outputs), and stays fully interpretable in measured units. The cost is that you need that count-aware loss — do **not** put raw counts under plain MSE.

Which is more reasonable? Match the target to the loss:

- Generic regressor with squared-error loss → predict **CLR** (use `axis=1`) for a leakage-free per-cell target, or fit-on-train (`axis=0`). This is the pragmatic default for most RNA→protein regression setups.
- Probabilistic / NB-Poisson decoder → predict **raw counts** with library-size handling. This is the more principled choice when you have such a model, and it lets you derive normalized values afterward rather than committing to one normalization up front.

A useful tie-breaker: CLR-space metrics reward getting the *relative* profile right; count-space metrics reward absolute abundance. Evaluate in whichever space your downstream use actually cares about — and report it, since a model can look better or worse purely from which space the error is measured in.

6. Conventional defaults at a glance

Step	Function	Conventional value	Alternatives exposed
HVG count	<code>select_hvg</code>	2000	any <code>n_hvg</code>
HVG mode	<code>select_hvg</code>	<code>global</code> (prediction tasks)	<code>batch_aware</code> (for batch integration)
RNA norm	<code>preprocess_gex</code>	<code>normalize_total(1e4) + log1p</code>	any <code>target_sum</code>
RNA scaling	<code>scale_expression</code>	<code>global</code> , no clip	<code>by_batch</code> , set <code>max_value</code> to clip
ADT norm	<code>clr_transform</code>	CLR, <code>axis=0</code>	<code>axis=1</code>
ADT scaling	<code>preprocess_adt</code>	off	<code>global</code> / <code>by_batch</code> , no clip

7. Gotchas

- **Always start from** `layers["counts"]` for both HVG selection and CLR. Reusing a previously-normalized `.X` is the most common source of subtle, hard-to-trace bugs.
 - **CLR must run on raw counts, not on whatever is in** `.X`. It's easy to leave ADT carrying a stale, already-transformed `.X`; the pipeline explicitly resets ADT to `layers["counts"]` before CLR so the composition is computed from real counts.
 - **`scikit-misc` is required** for `flavor="seurat_v3"`; install it before running.
 - **Keep RNA and ADT cell order identical.** The notebook asserts this; don't filter cells in one modality without applying the same filter to the other.
 - **Pick CLR `axis` and scaling mode once** and document them. Mixing conventions across datasets makes any downstream comparison uninterpretable.
-

8. Outputs

```
python

adata_rna_proc.write("cite_nips_RNA_proc.h5ad")    # scaled log-norm HVG GEX
adata_adt_proc.write("cite_nips_ADT_proc.h5ad")    # CLR ADT
pd.DataFrame({"hvg_gene": hvg_genes}).to_csv("cite_nips_hvg_genes.csv")
```

Saving the HVG list alongside the matrices means a later split can reuse the exact same genes, and the `lognorm` / `clr` layers let you recover the pre-scaling state without re-running anything.